Parallelized online Bayesian structured RNN

Overview of the Learning Scheme

Parallelized data can be written as:

$$\{u_{j,t}\}_{j\in[0,1,\cdots,u.shape[0]-1],t\in[0,1,\cdots,u.shape[1]-1]}$$

- the number of discrete data points: $u.shape[1]$
- the number of time series data: $u.shape[0]$

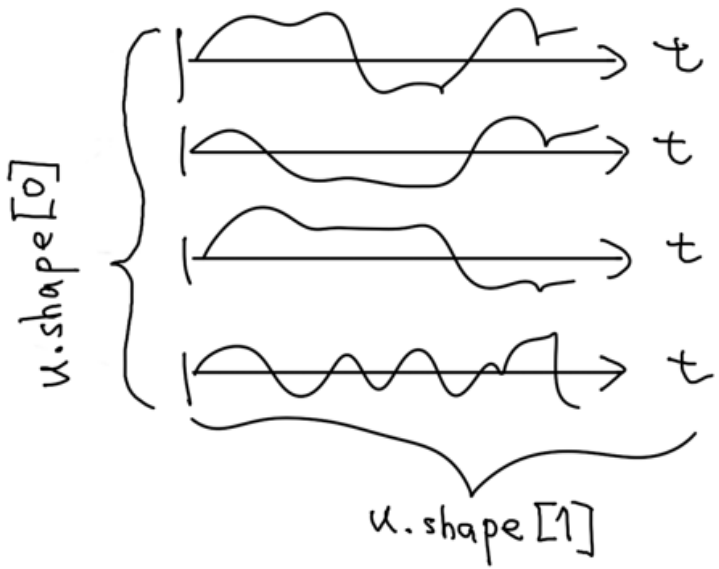


Figure: Image of data shape

Delay coordinate (see Nakai-Saiki 2021)

Rewrite the data in a form of *dim*-dimensional vector as the following:

$$U_{j,t} = \begin{pmatrix} u_{j,t} \\ u_{j,t-lag} \\ \vdots \\ u_{j,t-(dim-1)lag} \end{pmatrix}.$$

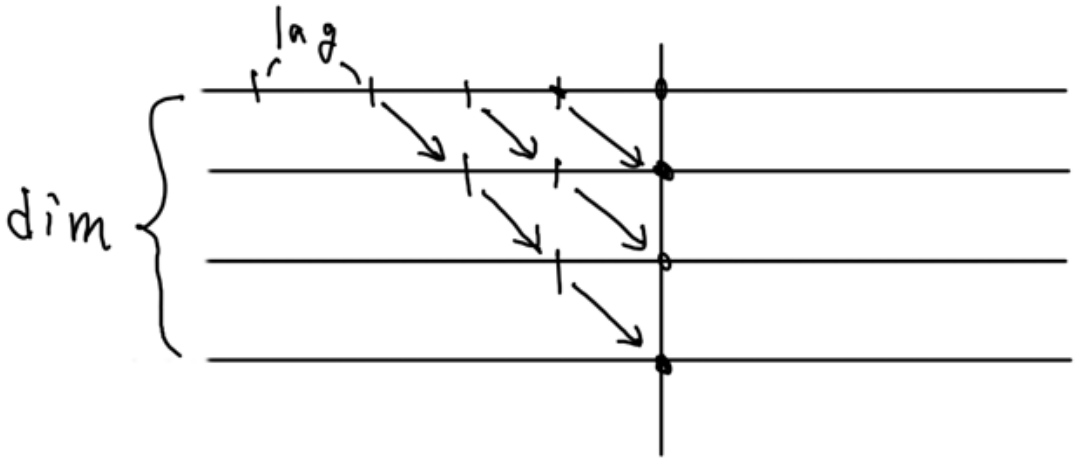


Figure: Image of $U_{j,t}$

RNN (overview)

Trainable parameters

- Input weight matrix: $W^{in} \in \mathbb{R}^{N_x \times dim}$
- Recurrent weight matrix: $W \in \mathbb{R}^{N_x \times N_x}$
- Output weight matrix: $W^{out} \in \mathbb{R}^{dim \times N_x}$

Substitute data $U_{j,t} \in \mathbb{R}^{dim}$ and feature vector $x_{j,t} \in \mathbb{R}^{N_x}$
Output predicted value $\widehat{U}_{j,t+1} \in \mathbb{R}^{dim}$ and feature vector $x_{j,t+1} \in \mathbb{R}^{N_x}$

$$(U_{j,t}, x_{j,t}) \mapsto (\widehat{U}_{j,t+1}, x_{j,t+1}),$$

$$\begin{cases} x_{j,t+1} & = (1-\alpha)x_{j,t} + \alpha \tanh(W^{in}U_{j,t} + Wx_{j,t}), \\ \widehat{U}_{j,t+1} & = W^{out}x_{j,t+1}. \end{cases}$$

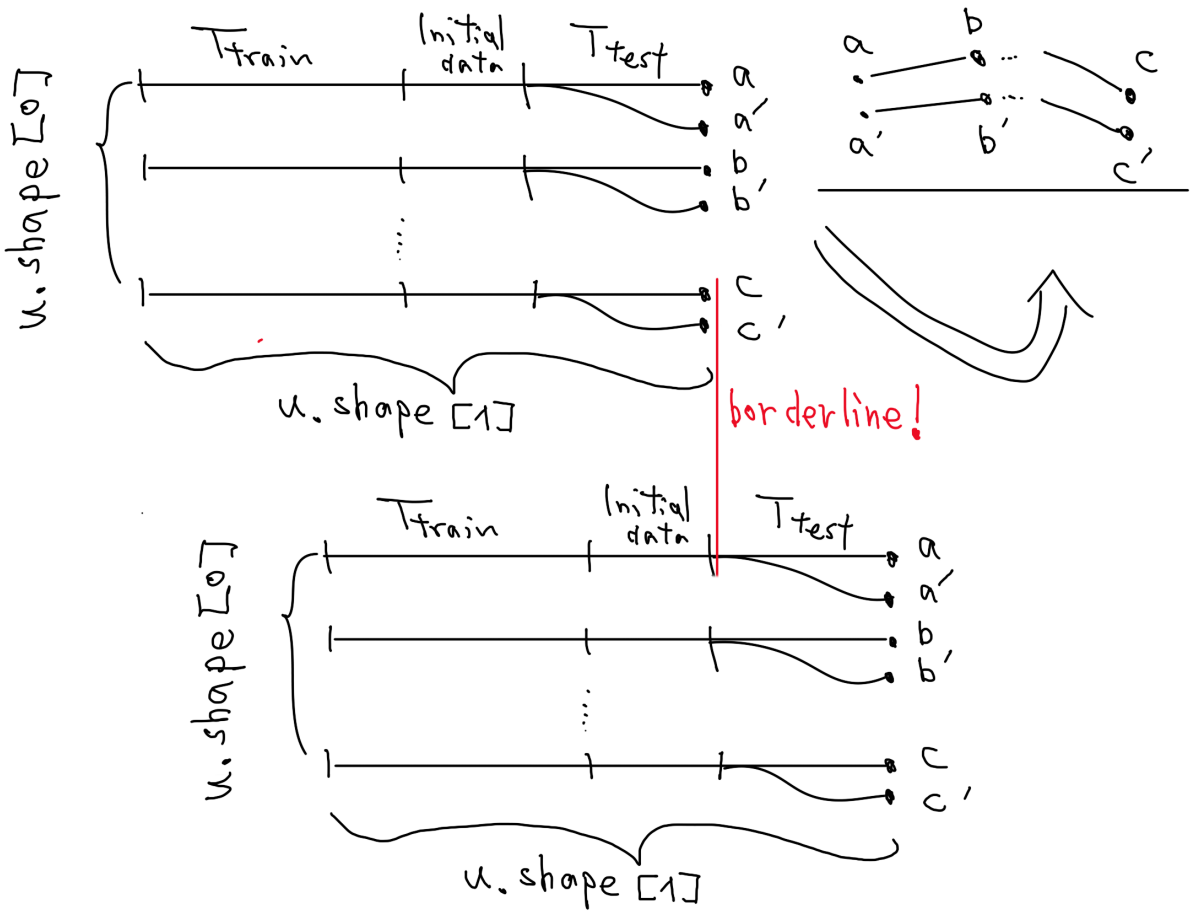
$\alpha \in (0, 1]$ is called the leaking rate, which is similar to the well-known concept of residual connections.

Conceptual diagram of the learning scheme

The scheme has two stages:

- Model selection
- Generalization performance evaluation

Each of them are including a training phase



Training phase

For each dim, lag, α, W^{in} and W, we first select W^{out} that minimizes the following E:

Evaluation function

$$E = \frac{1}{2} \sum_{j=0}^{u.shape[0]-1} \sum_{t=0}^{T_{train}-1} |U_{j,t} - W^{out} x_t|^2 + \frac{\beta}{2} \sum_{i=0}^{dim-1} \sum_{j=0}^{N_x-1} |W_{ij}^{out}|^2.$$

- $\beta > 0$ is a regularization parameter
- T_{train} is the number of training data points
- due to convexity, W^{out} can be solved analytically

Model selection phase

- We forecast T_{test} steps ahead
- The initial value for model selection is $U_{j,T_{train}}$
- $x_{j,T_{train}}$ is already obtained during the training phase

Based on these, the next step is estimated as follows:

$$\begin{cases} x_{j,T_{train}+1} & = (1-\alpha)x_{j,T_{train}} + \alpha \tanh(W^{in}U_{j,T_{train}} + Wx_{j,T_{train}}), \\ \widehat{U}_{j,T_{train}+1} & = W^{out}x_{j,T_{train}+1}. \end{cases}$$

The resulting $\widehat{U}_{j,t}, x_{j,t}$ are then sequentially fed into the following recurrence relation to generate the predicted data:

$$\begin{cases} x_{j,t+1} & = (1-\alpha)x_{j,t} + \alpha \tanh(W^{in}\widehat{U}_{j,t} + Wx_{j,t}), \\ \widehat{U}_{j,t+1} & = W^{out}x_{j,t+1} \end{cases}$$

for $t = T_{train}+1, \cdots, T_{train}+T_{test}$.

Bayesian optimization phase

Apply **Bayesian opt (Optuna)** using the prediction data $\widehat{U}_{j,T_{train}+T_{test}}$.
More specifically, minimize the following MAE-based evaluation function:

Evaluation function

$$\frac{1}{u.shape[0]} \sum_{j=0}^{u.shape[0]-1} |\widehat{U}_{j,T_{train}+T_{test}} - U_{j,T_{train}+T_{test}}|$$

in the selection of the following parameters:

dim, lag, α, W, W^{in}.

Generalization Performance: re-training Phase

After selecting dim, lag, α, W, and W^{in} in the previous phase, we implement a new test to evaluate generalization performance. We re-select W^{out} that minimizes the following E:

Evaluation function

$$E = \frac{1}{2} \sum_{j=0}^{u.shape[0]-1} \sum_{t=T_{test}}^{T_{train}+T_{test}-1} |U_{j,t} - W^{out} x_{j,t}|^2 + \frac{\beta}{2} \sum_{i=0}^{dim-1} \sum_{j=0}^{N_x-1} |W_{ij}^{out}|^2.$$

- We evaluate the generalization performance from $T_{train} + T_{test}$
- The initial value is $U_{j,T_{train}+T_{test}}$
- $x_{j,T_{train}+T_{test}}$ is already obtained during the re-training phase

Generalization Performance: evaluation phase

We generate the next step as follows.

$$\begin{cases} x_{j,T_{train}+T_{test}+1} & = (1-\alpha)x_{j,T_{train}+T_{test}} \\ & \quad +\alpha\tanh(W^{in}U_{j,T_{train}+T_{test}} + Wx_{j,T_{train}+T_{test}}), \\ \widehat{U}_{j,T_{train}+T_{test}+1} & = W^{out}x_{j,T_{train}+T_{test}+1}. \end{cases}$$

The $(\widehat{U}_{j,t}, x_{j,t})$ obtained in this manner are then sequentially fed into the following recurrence relation to generate the prediction data:

$$\begin{cases} x_{j,t+1} & = (1-\alpha)x_{j,t} + \alpha\tanh(W^{in}\widehat{U}_{j,t} + Wx_{j,t}), \\ \widehat{U}_{j,t+1} & = W^{out}x_{j,t+1} \end{cases}$$

for $t = T_{train} + T_{test} + 1, \cdots, T_{train} + 2T_{test}$. Then, finally, the generalization performance is evaluated using the following MAE:

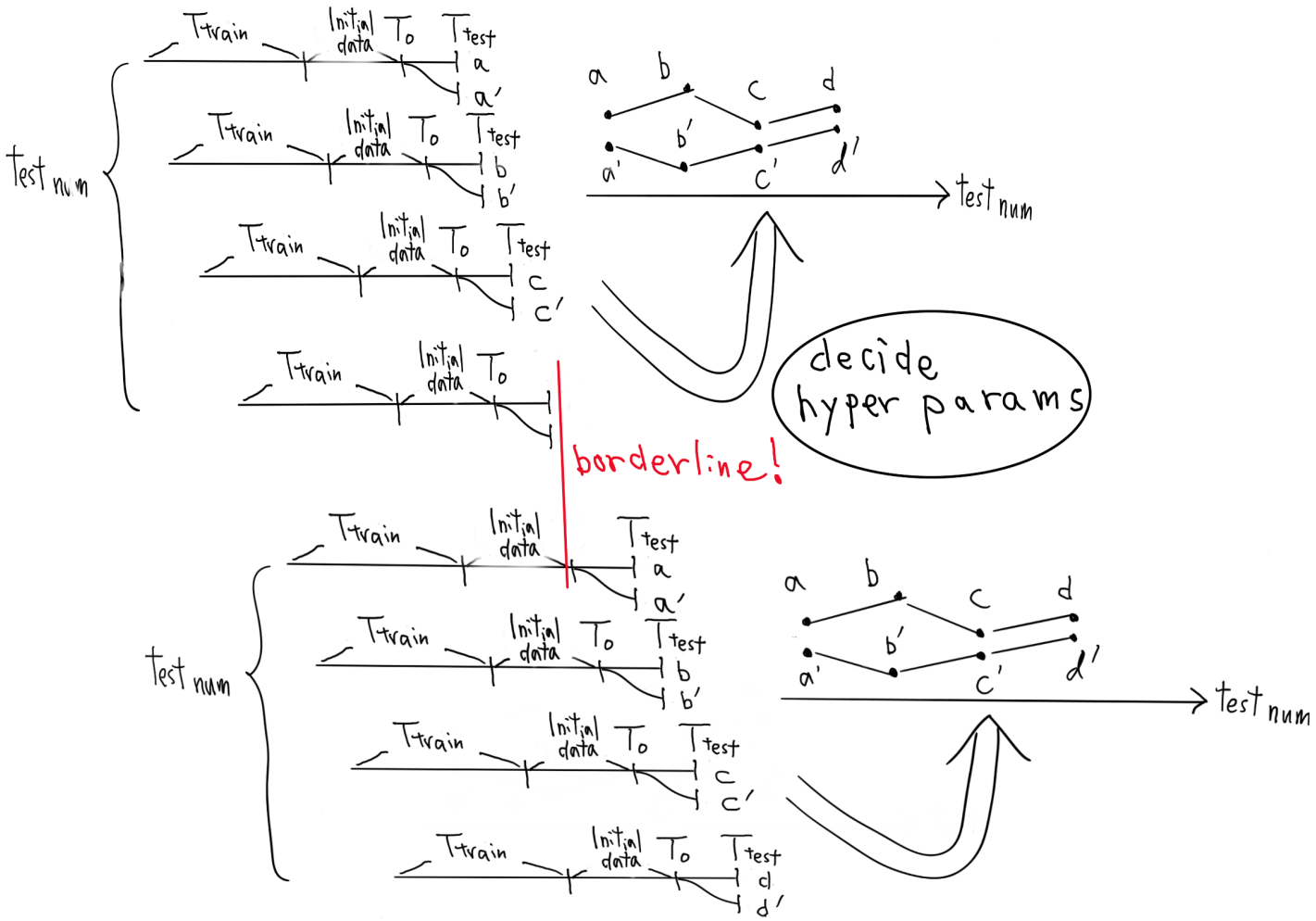
Evaluation function

$$\frac{1}{u.shape[0]} \sum_{j=0}^{u.shape[0]-1} |\widehat{U}_{j,T_{train}+2T_{test}} - U_{j,T_{train}+2T_{test}}|$$

ML Results
(various pipelines)

Learning framework: ElNino and MJO

Pearson correlation coefficient (PCC) between real data and prediction



Generalization performance evaluation: $T_{test} = 1, 2, \cdots$

Threshold of the correlation is 0.5 in this competition!!

El Nino prediction (Jinno-Mitsui-Nakai-Saiki-Y 2025)

Nino 3.4 (this time series expresses the strength of El Nino)

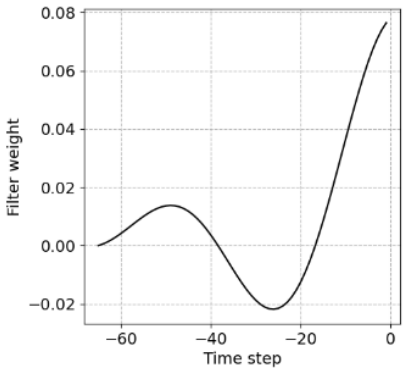
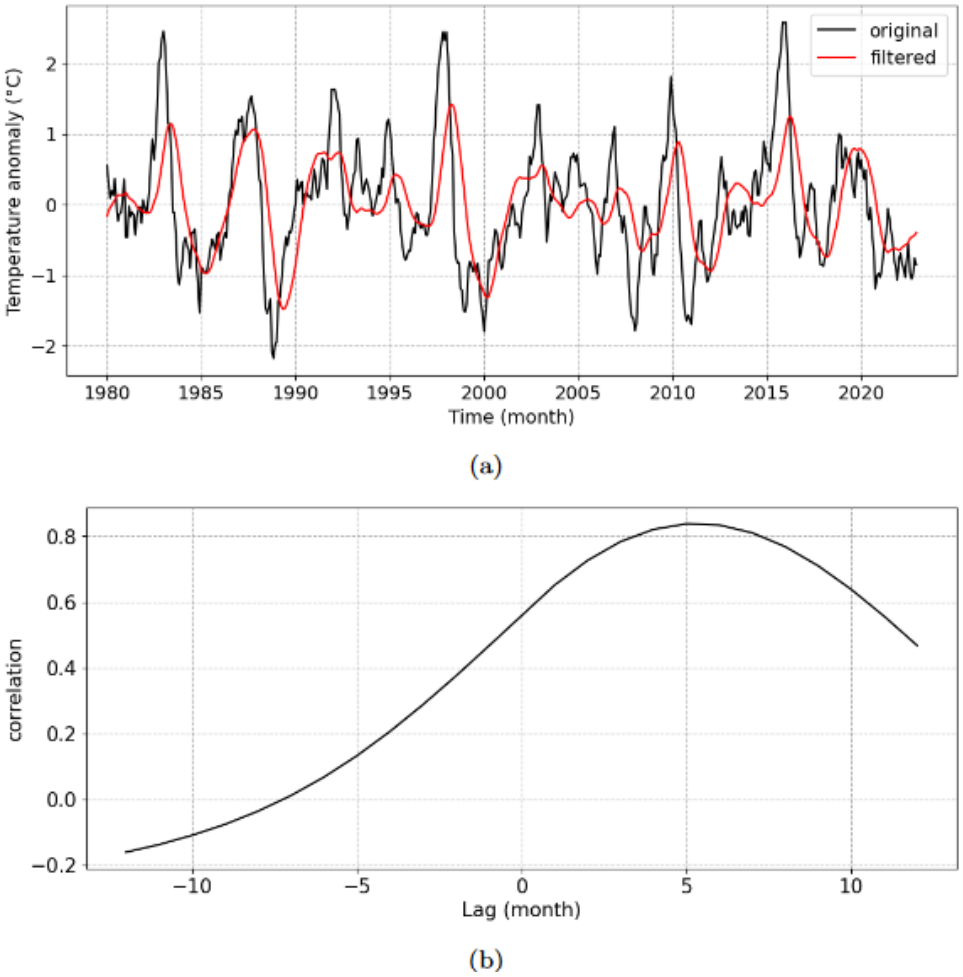


Figure: data-driven filter



(a): original (black) and filtered (red)

(b): Lag correlation between original and filtered : correlation > 0.8

Result (El Nino)

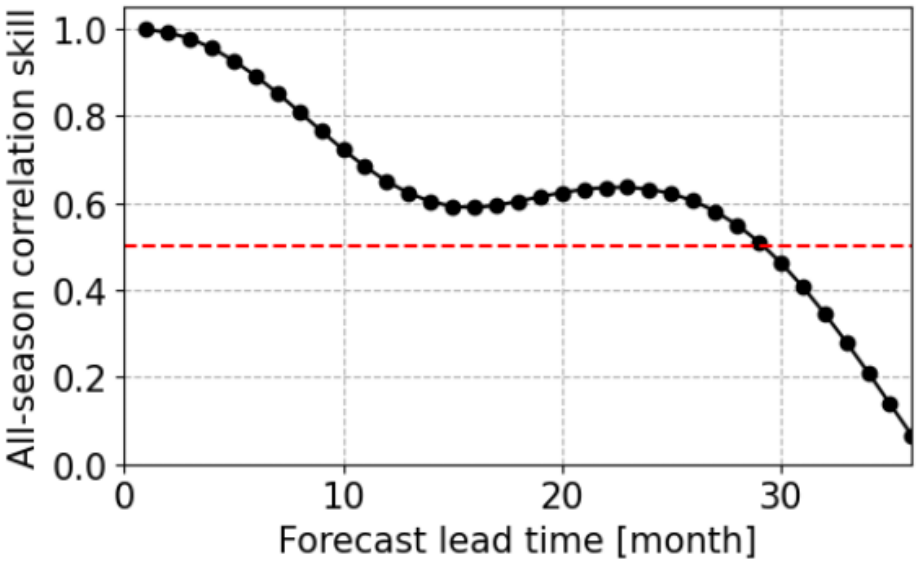


Figure: Prediction with our novel filter
(29 minus 5(lag)$=24$ months)

- Reservoir Hassanibesheli-Kurths-Boers (2022): 19 months (but incorporating future values due to the band-pass filter)
- CNN Ham-Kim-Luo (2019): 17 months
- Physical model SINTEX-F: 12 months

Result (MJO prediction)

Nakai-Suematsu-Miura-Y-Takasuka-Jinno-Saiki (arXiv)

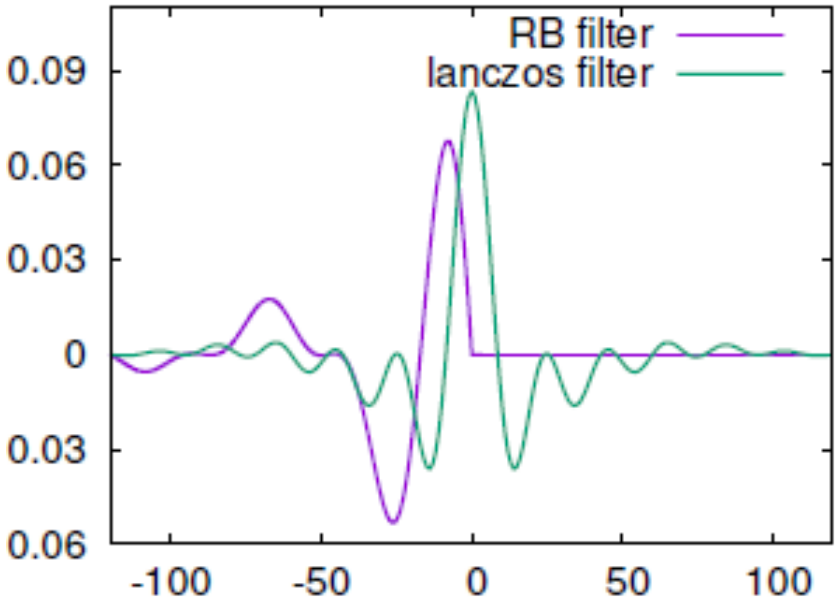


Figure: Lag (weighted center): -7.9

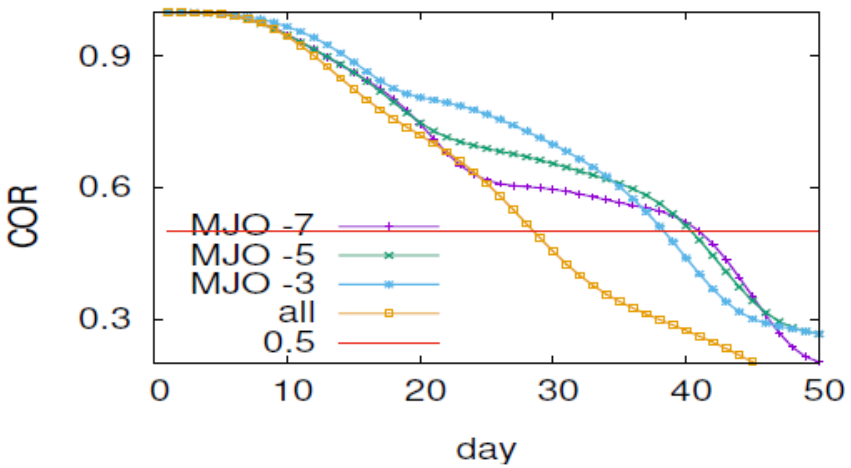
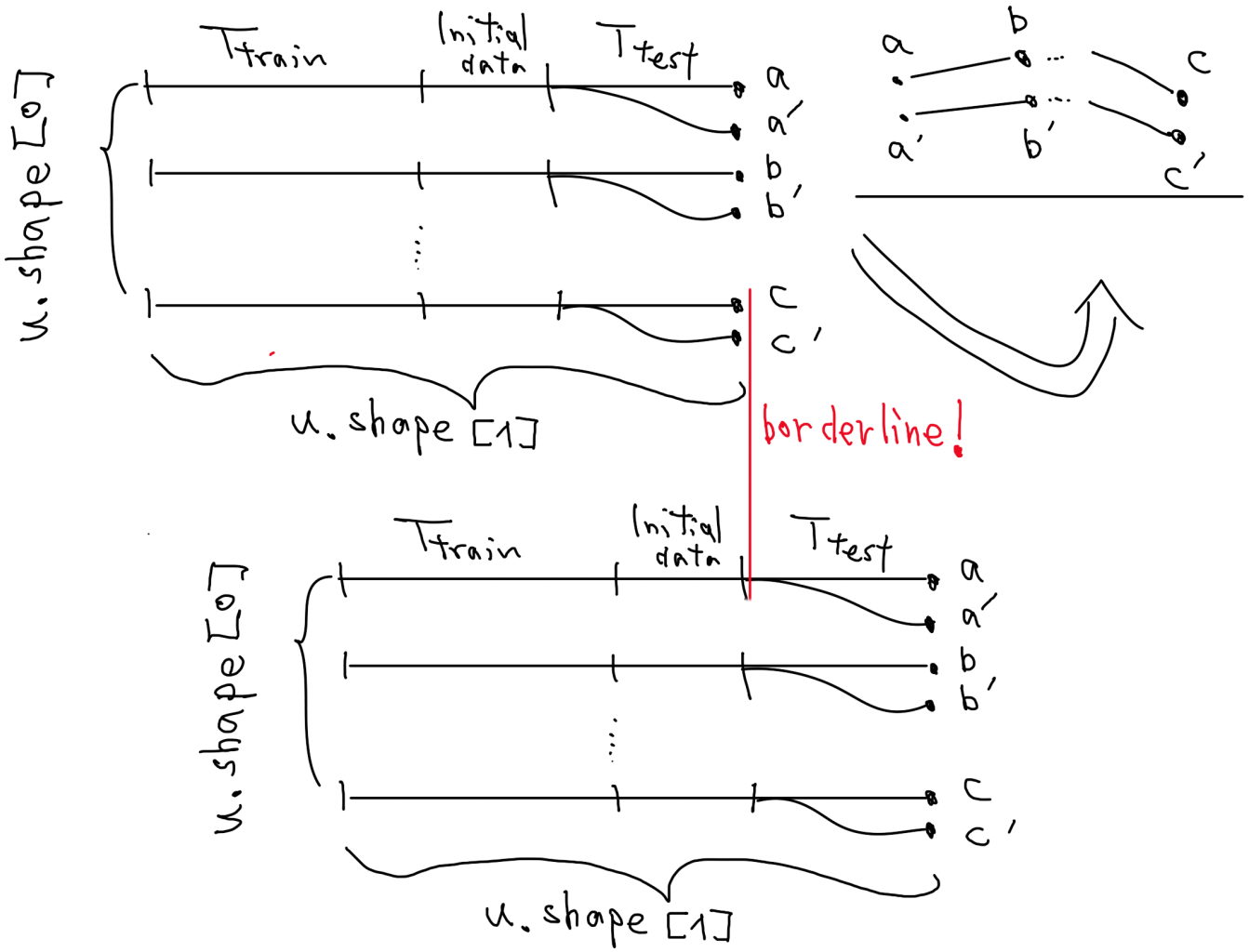


Figure: Pred with our RB filter (29 minus 8(lag)=21)

Even when using state-of-the-art physical models with high computational cost, the prediction accuracy is only on the order of about 20 days!

Learning framework: Tokyo WindSpeed and SP500

Mean absolute error (MAE) between real data and prediction

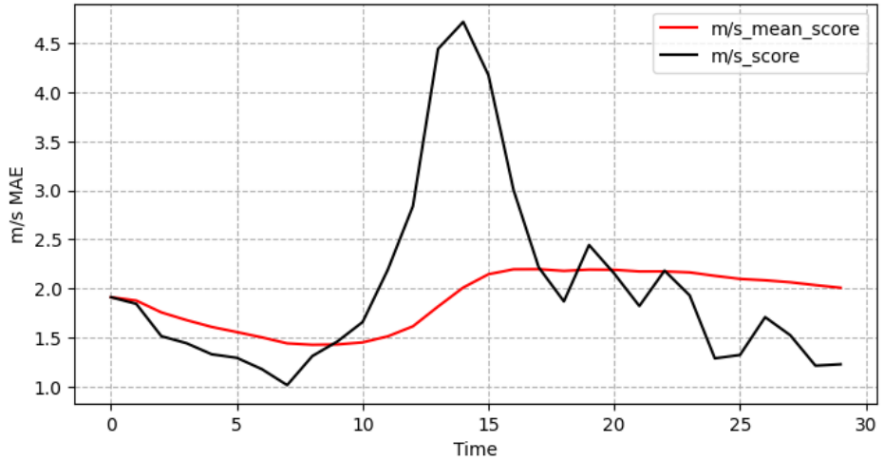


Generalization performance evaluation: choose one T_{test}

Result (windspeed in Tokyo region)

- predict 3 hours ahead ($T_{test} = 3$)
- most recent 1,000 hours of data are used to train($=: T_{train}$)
- Thirty independent trials were performed using a rolling-window validation scheme

Result MAE: 2.01 m/s (data-driven filter is applied: correlation 0.83, Jinno-Mitsui-Nakai-Saiki-Yoneda 2025)



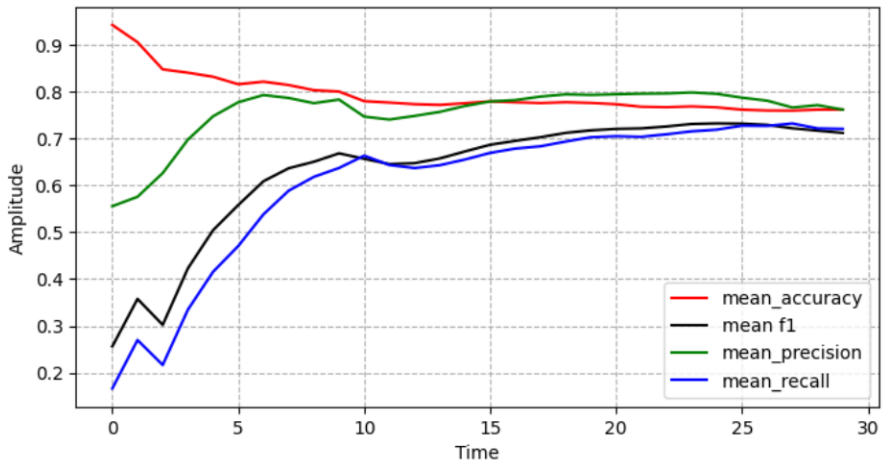
Cheng and Steenburgh (2005) CIRP, WRF (physical models)

MAE: 1.6 $\sim$ 1.9 m/s, but unclear whether a filter is applied...

Result (stock S&P500)

- predict 1 day ahead ($T_{test} = 1$)
- most recent 60 days of data are used to train($=: T_{train}$)
- learnable params $\approx$ 1,000
- Thirty independent trials were performed using a rolling-window validation scheme

Result Accuracy 0.762, F1 0.712, Recall 0.721 (data-driven filter is applied: correlation 0.985, Jinno-Mitsui-Nakai-Saiki-Yoneda 2025)

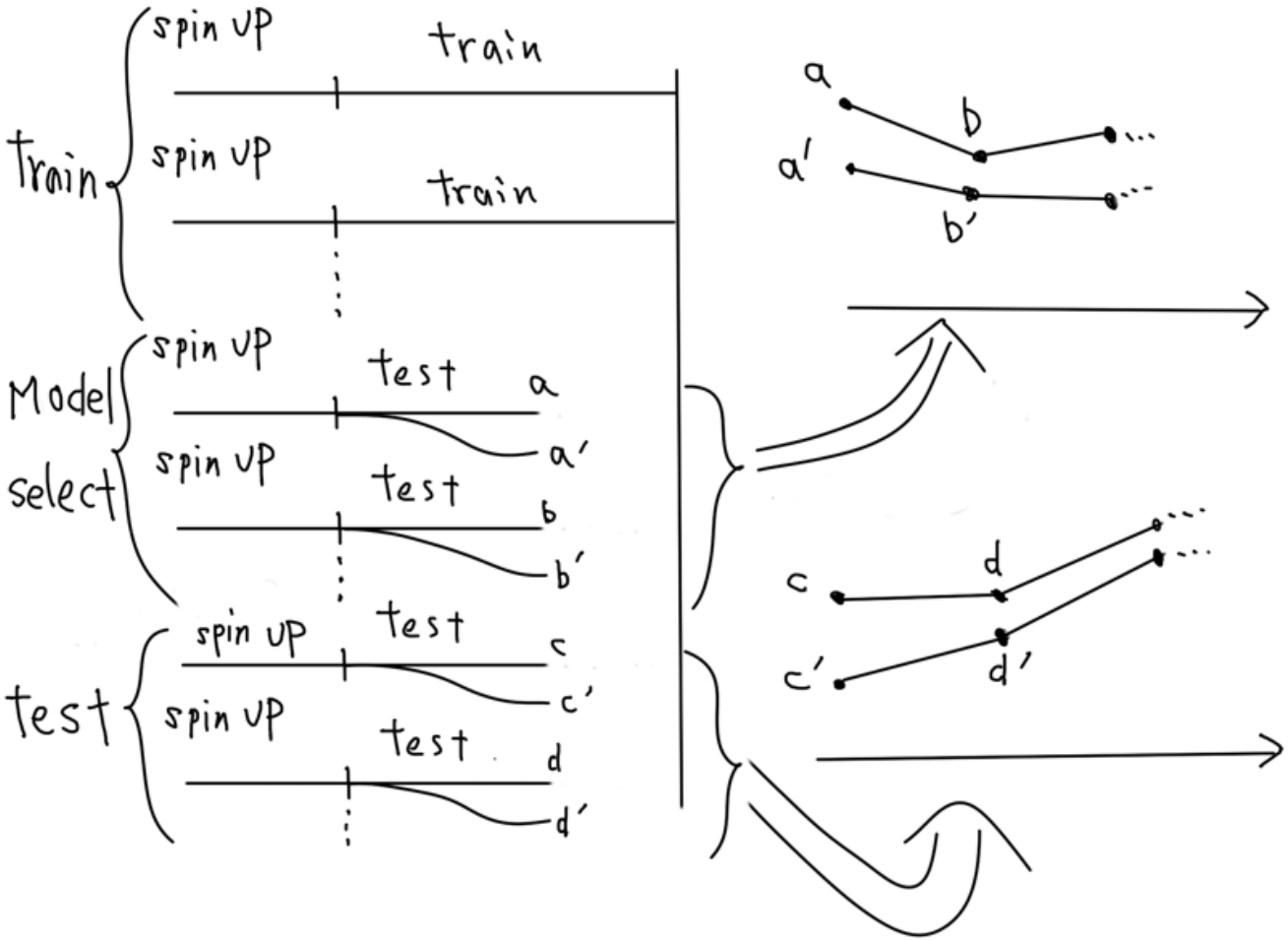


Gil, Duhamel-Sebline, McCarren (2024) xLSTM-TS

Accuracy 0.709, F1 0.730, Recall 0.768, wavelet filter is applied.
learnable params = 125,389, very huge!!

Learning framework: Lorenz 63

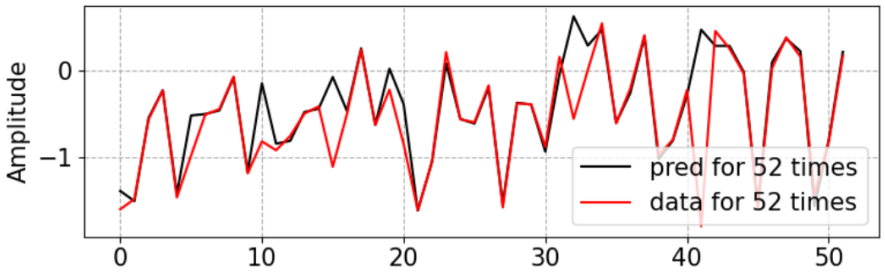
Mean absolute error (MAE) between real data and prediction



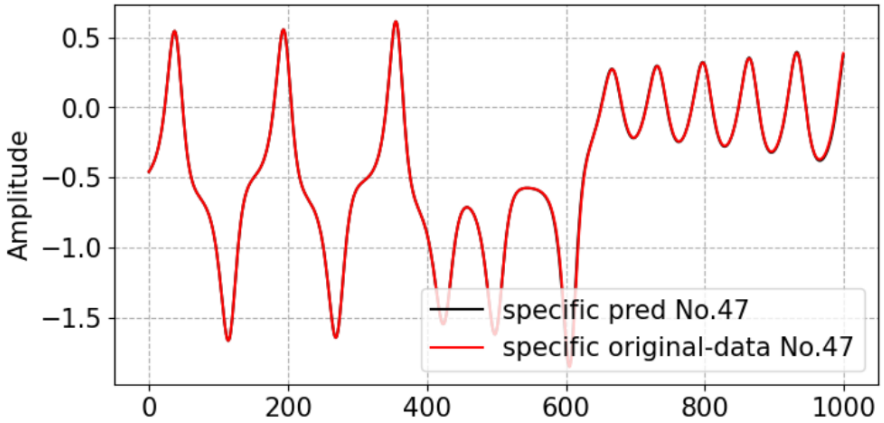
Generalization performance evaluation: take average between 1 and T_{test}

Result (Lorenz system)

- training dataset$\approx 600,000 \times 3$
- predict 1000-steps ahead ($T_{test} = 1000$), nondimensional time of 10
- standardized dataset
- MAE as the mean error over the entire prediction horizon



1000-step-ahead pred vs. ground truth



ParallelReservoir: mean-MAE 0.014,
Prediction: 1000 steps ahead

Feng et al. (2025) Physics-Guided Learning (PGL) method

A standardized mean-MAE of approximately $0.10 \sim 0.12$.

- training dataset$\approx 6,000 \times 3$
- incorporating physical laws as gentle hints, as a softer way of integrating physics compared to PINNs.

Thank you!